yassai

Yet Another Super Simple AI agent
for the AMD Developer Hackathon Act 2

100%
Task accuracy (40/40 on benchmark)

71%
Token reduction via batching

~48 MB
In-process classifier (int8 ONNX)

~5 ms
Classifier latency per task

Golang | Fireworks (minimax-m3) | MicroPython + Wazero WASM | ModernBERT classifier | Trained on AMD ROCm | OpenAI SDK

Ranked by fewest Fireworks tokens subject to an accuracy gate. Design goal: be correct first, then spend as few tokens as possible getting there.

Code as Agent Harness

The loop

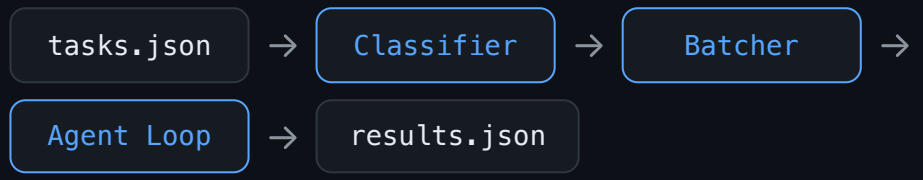
- → Model emits fenced `micropy` code blocks
- → Host extracts, runs them in a MicroPython WASM sandbox (wazero)
- → Observations fed back as user messages
- → Loop continues until `submit()` is called
- → No turn limit, no JSON parsing as primary path
- → State persists across code blocks in `vars`

```
# Compute, store, submit r1 = sh.run(command='python3 -c
"print(17*23)"') vars['t1'] = r1['output'].strip()
submit(answers=[ {'task_id': 't1', 'answer': vars['t1']}
])
```

Available tools (sandbox globals)

submit() · sh.run() · fs.read/write/edit/list · web.fetch/search · vars · tools.help()

Pipeline



Grounded in *Hitchhiker's Guide to Agentic AI* and *Code as Agent Harness Survey*. The agent reasons freely, builds state in `vars`, and computes answers programmatically instead of guessing.

In-Process Task Routing Classifier

Model

- → Backbone: IBM Granite Embedding R2 (ModernBERT, ~47M params)
- → Fine-tuned as 8-way multi-label head (sigmoid + per-label thresholds)
- → Exported to int8 ONNX, runs in-process on CPU via Go
- → Pure-Go BPE tokeniser (verified byte-identical to HuggingFace)
- → Head+tail truncation to 1024 tokens (leading or trailing instruction survives)
- → **Trained on AMD ROCm notebook**

8 capability categories

factual_knowledge · mathematical_reasoning · sentiment_classification · text_summarisation · named_entity_recognition · code_debugging · logical_deductive_reasoning · code_generation

Accuracy (held-out hard real-benchmark)

METRIC	RESULT
micro-F1	**0.977**
macro-F1	0.967
subset accuracy	0.930
prev. MiniLM v1 (micro-F1)	0.889

Training data

- → 14,388 synthetic + real prompts (7,142 train / 400 val / 199 test)
- → Independent blind re-label adopted as ground truth to remove noise
- → Sources: Umans synthetic gen, real benchmark fetch, easy short prompts

Why it pays for itself

Adding predicted categories to the prompt costs ~10 tokens per task but nets **~12% fewer Fireworks tokens** overall, because the model reasons less when told the task type up front.

TOKEN EFFICIENCY

Batching and Adaptive Routing

Batch size vs token cost (40 tasks)

BATCH SIZE	CALLS	TOKENS	ACCURACY
1 (no batching)	42	78,865	95%
10	4	31,985	100%
20	2	**23,141**	**100%**

Batching amortises the fixed per-call prompt overhead (~260-token system prompt) over many tasks. Fewer, fuller batches = fewer tokens.

Adaptive reasoning effort

- → Tasks grouped by capability tier; one effort level per batch
- → Only logical/deductive puzzles get `xhigh`
- → Everything else at `low` (already at ceiling)
- → Category hints injected only for categories present in batch
- → Memory and skills blocks omitted when empty (zero cost)

Token breakdown (3 hard math tasks, batched)

26,112
Total tokens

20,802
Cached (80% hit rate)

1,780
Reasoning tokens

Per-category token profile (routeprobe)

CATEGORY	EFFORT	TOKENS	PASS
factual	low	3,609	5/5
maths	high	4,123	5/5
sentiment	low	3,789	5/5
NER	low	8,442	5/5
summarisation	low	8,193	5/5
logical_reasoning	high	21,835	5/5
code_debugging	high	6,914	5/5
code_generation	high	6,400	5/5

What is Left on the Table

Smarter routing across models

Currently all tasks go to one model (minimax-m3). The classifier already tags capability; routing factual/sentiment to a cheaper model and logical/code to a stronger one could cut tokens further while maintaining accuracy.

Cost / quality / speed tradeoff dial

The adaptive effort tier is binary (low vs xhigh). A continuous per-category effort slider, tuned from routeprobe data, would find the Pareto frontier of accuracy vs token cost for each category.

Load-bearing vs decorative classification

The classifier currently feeds categories to the prompt. It could also drive batch composition (group similar-difficulty tasks), max_tokens allocation, and concurrency priority. Right now only the effort tier uses it.

Fine-tuning for compact reasoning (Grug-style)

Base model reasoning is verbose (1,780 reasoning tokens for 3 tasks). A Grug-style compact-reasoning LoRA fine-tune on agent trajectories could halve reasoning tokens while preserving code-block quality. Fireworks managed LoRA ($50 credits available) on Gemma 4 26B-A4B / 31B.

Caching and context compression

80% cache hit is already strong, but prompt-prefix caching could be improved further by stabilising batch ordering. Context compression (pxpipe-inspired) for long agent loops would reduce per-turn growth.

Classifier ensemble and confidence routing

Low-confidence classifications could trigger a fallback (second model or majority vote). Currently best-effort with no confidence threshold.